

Savitribai Phule Pune University

Modern Education Society's Wadia College of Engineering, Pune

19, Bund Garden, V.K. Joag Path, Pune – 411001.

ACCREDITED BY NBA AND NAAC WITH 'A++' GRADE

DEPARTMENT OF COMPUTER ENGINEERING

A

MINI PROJECT REPORT

ON

“Design and Develop a Blockchain-Based Decentralized E-Voting System”

VoteChain – Decentralized E-Voting System

B.E. (COMPUTER)

SUBMITTED BY

Name of Student	PRN	Roll No.
Yashvant Dayanand Mane	F24121004	68

GUIDED BY

Dr. S. R. Khonde

(Academic Year: 2026–2027)

DEPARTMENT OF COMPUTER ENGINEERING

MODERN EDUCATION SOCIETY'S WADIA COLLEGE OF ENGINEERING, PUNE

Modern Education Society's Wadia College of Engineering, Pune
Department of Computer Engineering

CERTIFICATE

This is to certify that the Mini Project report entitled “**Design and Develop a Blockchain-Based Decentralized E-Voting System**” (VoteChain) submitted by **Yashvant Dayanand Mane** (PRN **F24121004**, Roll No. **68**) is a bonafide work carried out by him and submitted during the 2026–27 academic year, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF ENGINEERING** in **COMPUTER ENGINEERING** of Savitribai Phule Pune University, at MESWCOE, Pune.

Dr. S. R. Khonde
Project Guide

Date :
Place : Pune

Sign :
Head of the Department
Department of Computer Engineering

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my respected guide, **Dr. S. R. Khonde**, for providing valuable guidance, constructive suggestions and continuous support throughout the development of my mini project on “**Design and Develop a Blockchain-Based Decentralized E-Voting System**”.

I am also thankful to the Department of Computer Engineering, Modern Education Society's Wadia College of Engineering, Pune, for providing the necessary facilities, resources and academic support required for successfully completing this project. I sincerely appreciate the guidance and assistance provided by the faculty members throughout the project.

Finally, I thank my family and friends for their encouragement during this work.

Yashvant Dayanand Mane

PRN: F24121004 · Roll No. 68

B.E. Computer Engineering

ABSTRACT

VoteChain is a decentralized electronic voting application in which the complete election is executed by an Ethereum smart contract. An administrator (the contract owner) creates an election, adds candidates, registers the wallet addresses of eligible voters and opens and closes voting. A registered voter connects MetaMask, verifies eligibility, reviews the candidates and casts exactly one vote. The vote is a signed blockchain transaction; the contract validates the election status, voter registration, double voting and the candidate id before updating the on-chain vote count and emitting a `VoteCast` event.

Unlike conventional online voting systems, VoteChain has no central database. The blockchain is the single source of truth for registered voters, candidates, voting status, vote counts and election status, so the results shown on the dashboard can be verified independently by anyone. The system is built with Solidity 0.8.24 and OpenZeppelin `Ownable`, tested with Hardhat (24 automated test cases, all passing), and a React + Vite + TypeScript frontend using ethers.js v6, Tailwind CSS and Recharts. The application includes a voter dashboard, live results with charts, an admin dashboard and a built-in blockchain explorer that shows the transaction hash, block number, status, timestamp, sender and contract address of every action.

The report also discusses the difference between transparency and voter privacy: wallet addresses and transactions on a public blockchain are visible, so votes are pseudonymous, not anonymous. VoteChain is an academic demonstration and is not intended for government elections, which additionally require verified identity, ballot secrecy, coercion resistance and legal infrastructure.

Keywords: blockchain, e-voting, Ethereum, smart contract, Solidity, MetaMask, ethers.js, transparency, tamper resistance, React.

TABLE OF CONTENTS

1	Introduction
2	Problem Statement and Objectives
	2.1 Objectives · 2.2 Scope · 2.3 Existing vs Proposed System
3	Why Blockchain?
4	System Architecture and Design
	4.1 Architecture · 4.2 DFD · 4.3 Use Case · 4.4 Interaction Flow · 4.5 Data Flow
5	Smart Contract Design (Voting.sol)
	5.1 Data Model · 5.2 Functions · 5.3 Access Control · 5.4 Algorithm · 5.5 Life-cycle
6	Frontend Architecture
7	Technology Stack and System Requirements
8	Testing
	8.1 Test Cases · 8.2 Gas Usage
9	Results and Output
10	Security Considerations, Transparency and Privacy
11	Limitations and Future Enhancements
12	Installation, Deployment and Demo Guide
13	Conclusion
14	References
A	Appendix: Mini Project Presentation Points

1. Introduction

Elections at every level, from a student council to a nation, depend on trust: voters must believe that only eligible people voted, that each person voted once and that the published count matches the ballots cast. In most online voting systems the votes are rows in a database controlled by the operator. Voters and candidates must simply trust that the operator, the server administrator or an attacker did not alter those rows.

Blockchain technology offers a different model. A smart contract is a program deployed on a blockchain whose code and state are replicated across many nodes and changed only by valid, signed transactions. Once a transaction is included in a block, changing it would require rewriting the chain, which is computationally and economically infeasible on a public network. VoteChain uses this property to run a complete election inside the `Voting` smart contract. The web application is only a window onto the contract: it never stores votes and never computes results on its own.

The project is designed as a polished, demonstrable final-year mini project. It runs on a local Hardhat blockchain (chain id 31337) or on the Sepolia public testnet, uses MetaMask for identity and transaction signing, and presents the election through a modern dark, glassmorphism-style dashboard.

2. Problem Statement and Objectives

Problem statement: Design and develop a secure, transparent and tamper-resistant electronic voting system using blockchain technology, in which eligible users connect their crypto wallet, verify their eligibility, view candidates, cast exactly one vote and see transparent election results, with all voting data stored and validated by a Solidity smart contract rather than a centralized database.

2.1 Objectives

- Implement a `Voting.sol` smart contract that manages elections, candidates, voter registration, voting and results.
- Restrict administrative operations to the election owner using OpenZeppelin `Ownable`.
- Guarantee one vote per registered wallet and reject invalid candidates and out-of-time votes on-chain.
- Emit events (`VoterRegistered` , `CandidateAdded` , `ElectionStarted` , `VoteCast` , `ElectionEnded`) for public auditing.
- Build a responsive React + TypeScript dApp with wallet connection, voter dashboard, results, admin and explorer pages.
- Show the full transaction life-cycle: MetaMask confirmation, pending state, block confirmation, hash and explorer link.

- Verify the contract with comprehensive automated tests and document the architecture, DFD, use cases and limitations.

2.2 Scope

The system targets small, low-stakes elections such as a student council, club or classroom poll, demonstrated on a local or test network. It covers the complete election life-cycle for one active election at a time, with the history of previous elections kept on-chain. Real-world identity verification (KYC), ballot secrecy and legal compliance are outside the scope and are discussed in Chapter 10.

2.3 Existing System vs Proposed System

Aspect	Existing / conventional e-voting	Proposed system – VoteChain
Vote storage	Rows in a central database controlled by the operator.	State variables of a smart contract replicated on every node.
Tamper resistance	An administrator or attacker with DB access can change counts.	Confirmed transactions cannot be altered; counts change only via <code>vote()</code> .
Double voting	Enforced by application code that users cannot inspect.	Enforced by the public contract (<code>AlreadyVoted</code>).
Result verification	Voters must trust the published report.	Anyone can call <code>getCandidates()</code> or replay <code>VoteCast</code> events.
Admin control	Server roles and passwords.	<code>onlyOwner</code> checked by the contract itself.
Audit trail	Server logs that can be edited or deleted.	Immutable event log with tx hash, block and timestamp.
Single point of failure	Central server / database.	Decentralized network of nodes (on a public chain).

3. Why Blockchain?

- **Immutability:** each block contains the hash of the previous block; changing an old vote would change every later hash and be rejected by the network.
- **Single source of truth:** every node computes the same contract state, so the results page, the admin page and an external auditor read identical numbers.
- **Programmable rules:** the voting rules (who, when, how many times, for whom) are code that executes the same way for everybody and cannot be skipped by the frontend.
- **Cryptographic identity:** a vote is valid only if it is signed by the private key of a registered address; nobody can vote on someone else's behalf without that key.
- **Public auditability:** events and transactions are permanently available; anyone can recount the election from the event log.
- **No trusted operator:** once the contract is deployed, even the administrator cannot change a vote or a count; the admin can only perform the actions the contract allows.

Important: these benefits concern *integrity* and *transparency*. They do not by themselves provide ballot secrecy or real-world identity. See Chapter 10.

4. System Architecture and Design

4.1 System Architecture

The architecture follows the flow **Frontend** → **MetaMask** → **ethers.js** → **Ethereum-compatible blockchain** → **Voting smart contract**. Write operations (vote, admin actions) are signed in MetaMask; read operations use a read-only JSON-RPC provider so that results are visible even without a wallet. Contract address, ABI, chain id and explorer URL are loaded from environment variables through a single configuration file, `contracts/config.ts`.

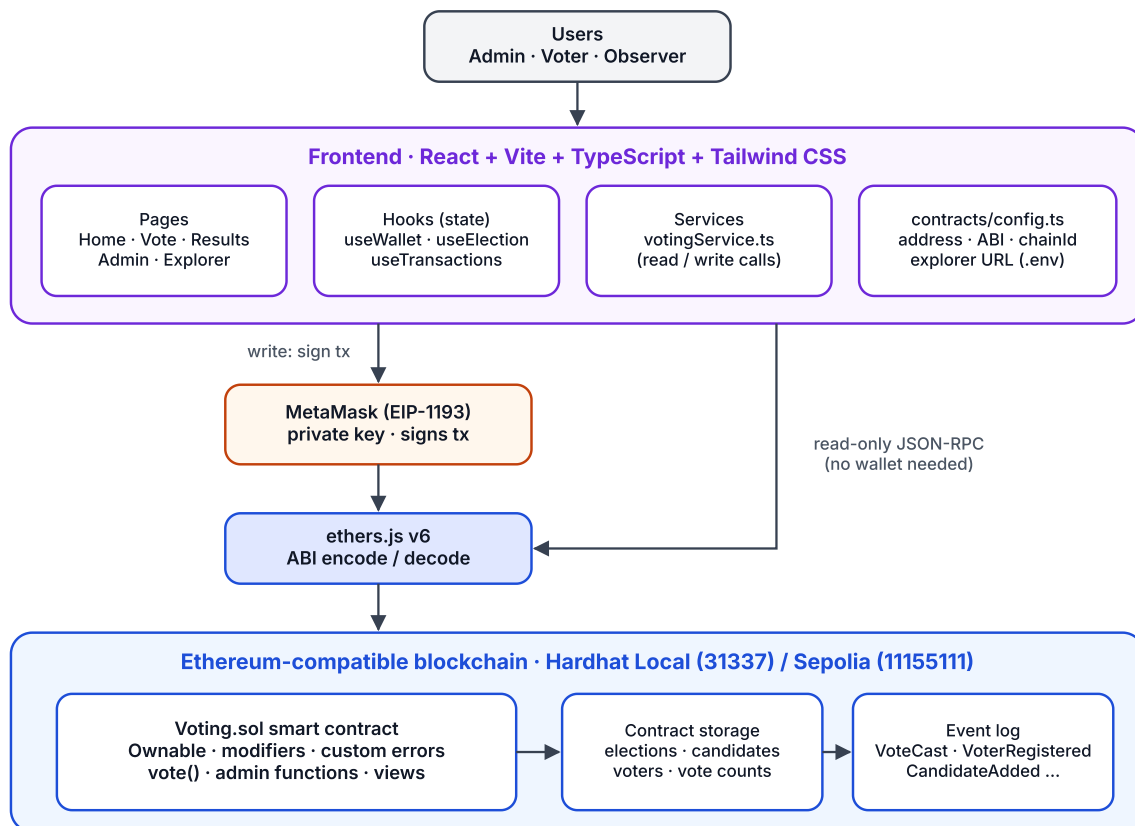


Figure 1: System architecture of VoteChain

4.2 Data Flow Diagrams

The Level 0 DFD shows VoteChain as a single process with three external entities. The Level 1 DFD decomposes it into six processes. All four data stores (D1–D4) are on-chain; there is no off-chain database.

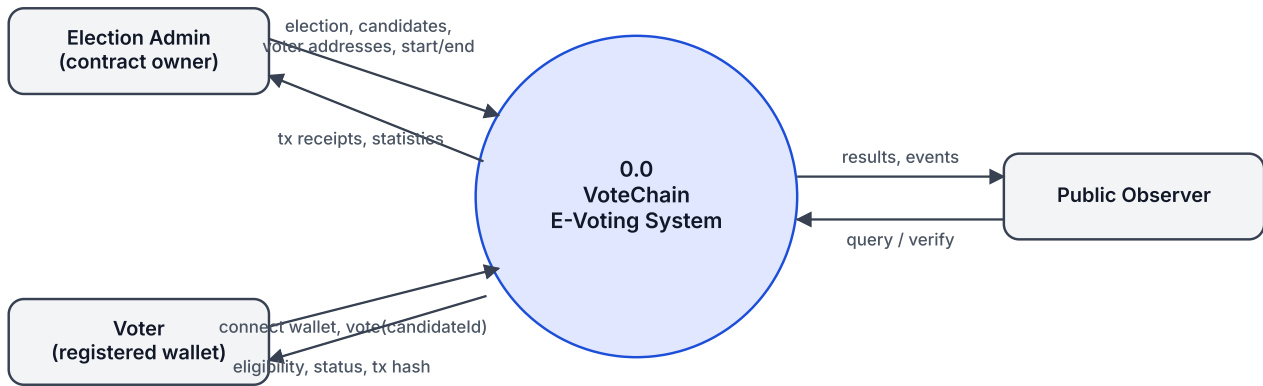


Figure 2: DFD Level 0 (context diagram)

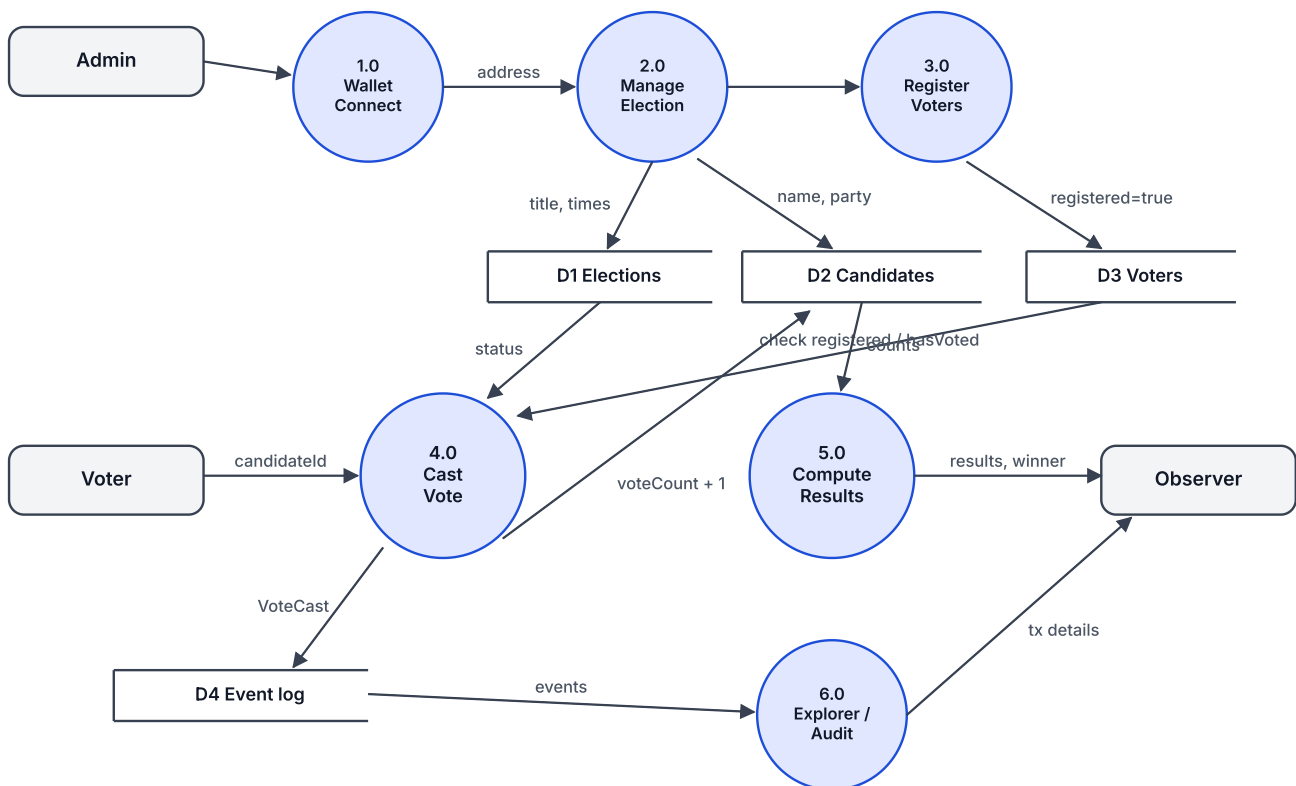


Figure 3: DFD Level 1

4.3 Use Case Diagram

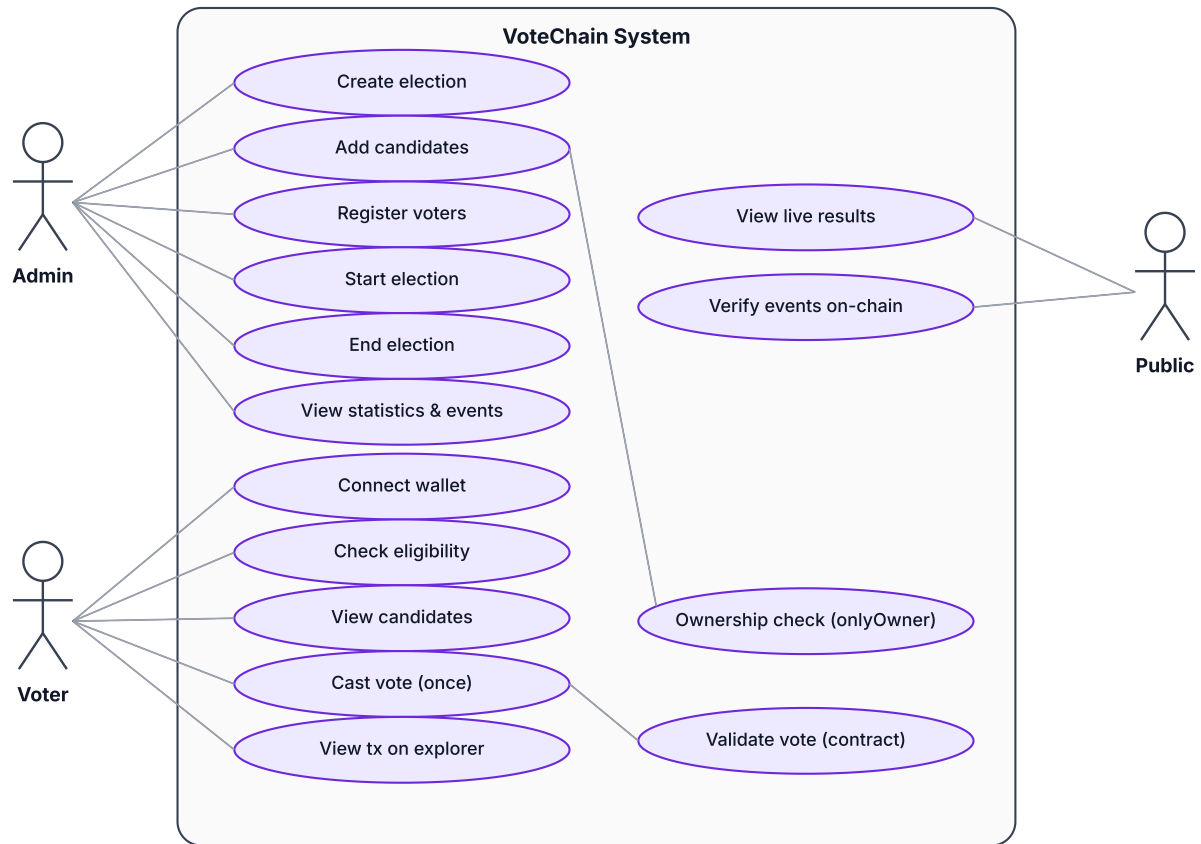


Figure 4: Use case diagram: Admin, Voter and Public observer

4.4 Smart Contract Interaction Flow (Casting a Vote)

Before MetaMask is opened, ethers.js estimates gas, which runs the contract checks in a dry run. An invalid vote (for example a second vote) is therefore reported to the user with a friendly message before any fee is paid.

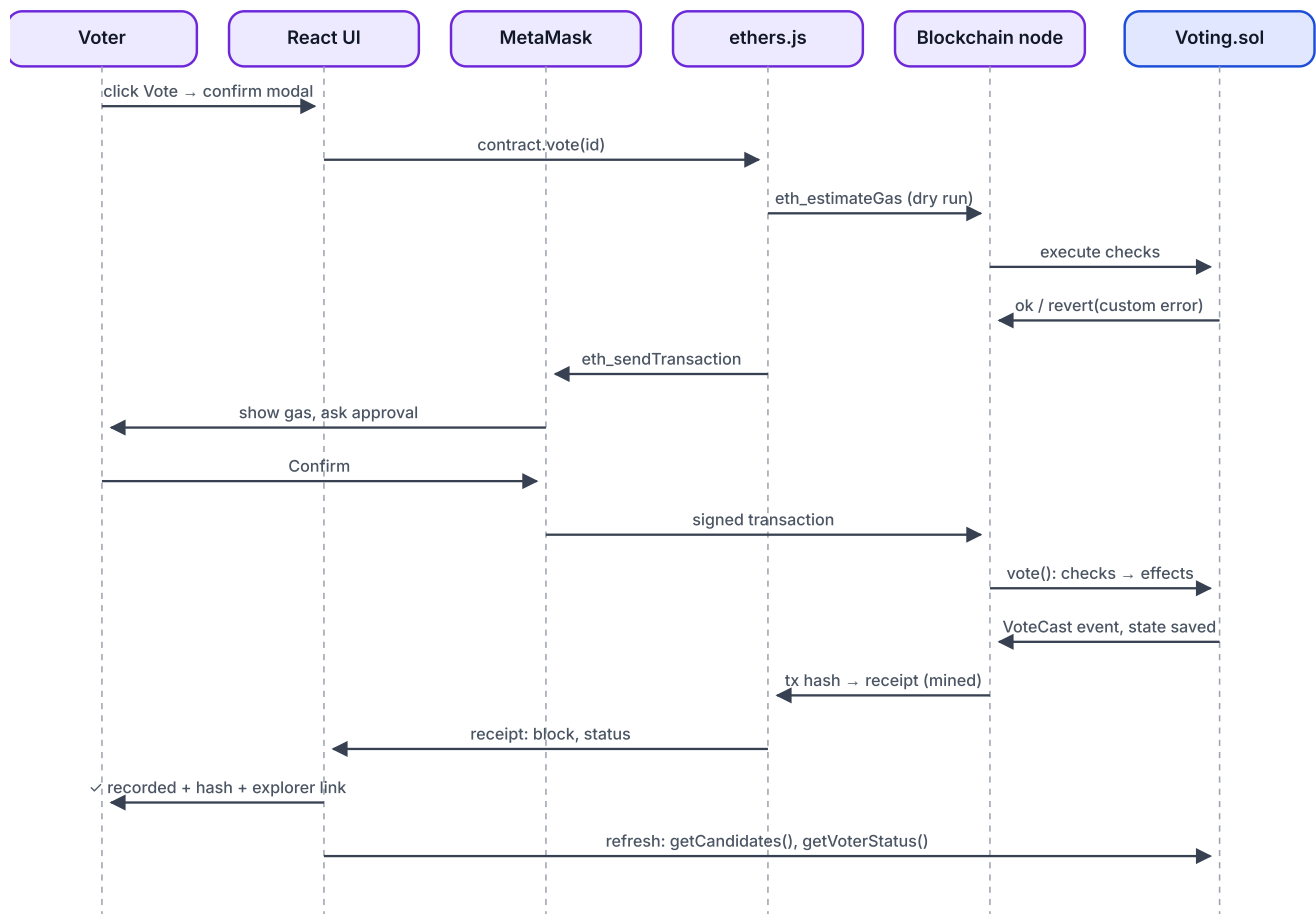


Figure 5: Sequence diagram of the vote transaction

4.5 Blockchain Data Flow Explanation

VoteChain replaces the traditional database with contract storage. Writes travel as transactions: the frontend builds the call with the ABI, MetaMask signs it, a node executes `Voting.sol` and the resulting state change is stored once the transaction is mined. Reads are free `eth_call` requests to view functions (`getElectionInfo`, `getCandidates`, `getVoterStatus`, `getWinners`). The frontend polls these every five seconds, so all dashboards stay in sync. Events are read with `eth_getLogs` from the deployment block and are used by the admin activity feed and the built-in explorer.

Data	On-chain location	Written by	Read by
Election (title, start, end, started, ended)	<code>_elections[id]</code>	<code>createElection / startElection / endElection</code>	<code>getElection(), getElectionInfo()</code>

Data	On-chain location	Written by	Read by
Candidates and vote counts	<code>_candidates[id][]</code>	addCandidate, vote	getCandidates(), getCandidate()
Voter registration and voted flag	<code>_voters[id][address]</code>	registerVoter(s), vote	getVoterStatus()
Totals	<code>registeredVoterCount</code> , <code>totalVotes</code>	registerVoter(s), vote	getElectionInfo()
Audit trail	Event log	every state-changing function	Explorer page, admin feed

5. Smart Contract Design (Voting.sol)

5.1 Data Model

Structure	Fields	Purpose
Candidate	uint256 id, string name, string party, string manifesto, uint256 voteCount	A candidate and their live on-chain vote count.
Voter	bool registered, bool hasVoted	Eligibility and one-vote enforcement per address.
Election	string title, uint256 startTime, uint256 endTime, bool started, bool ended	Status and timing of an election.
ElectionInfo	id, title, times, flags, votingOpen, candidateCount, registeredVoters, totalVotes	Summary returned to the dashboard in one call.

All mappings are keyed by an **election id**, so a new election starts with clean candidate and voter lists while previous results remain verifiable.

5.2 Functions

Function	Access	Description
createElection(title)	onlyOwner	Creates a new election (only if none exists or the last one ended).
addCandidate(name, party, manifesto)	onlyOwner, beforeStart	Adds a candidate with the next id (ids start at 1).
registerVoter(address) / registerVoters(address[])	onlyOwner	Authorizes one or many wallets; rejects zero and duplicate addresses.
startElection(durationSeconds)	onlyOwner, beforeStart	Opens voting; optional automatic deadline (0 = manual end).
endElection()	onlyOwner	Closes voting permanently and freezes results.
vote(candidateId)	registered voter, onlyWhileOpen	Casts exactly one vote and emits VoteCast.
getElectionInfo(), isVotingOpen()	public view	Status of the current election.
getCandidates(), getCandidate(id), getCandidatesOf(electionId)	public view	Candidates with vote counts (current or historic).
getVoterStatus(address)	public view	Registered / has-voted flags.
getWinners()	public view	Winning candidate id(s) and highest count; several ids mean a tie.

5.3 Access Control, Modifiers, Events and Errors

Administrative functions use OpenZeppelin's `onlyOwner`; the owner is the deploying account. Two custom modifiers enforce the election phase. Custom errors are used instead of revert strings to save gas; the frontend decodes them into readable messages.

```
modifier onlyWhileOpen() {
    if (currentElectionId == 0) revert NoElection();
    Election storage e = _elections[currentElectionId];
    if (!e.started) revert ElectionNotStarted();
    if (e.ended || (e.endTime != 0 && block.timestamp > e.endTime)) revert
    ElectionNotActive();
    -;
}
```

Events	Custom errors (revert reasons)
ElectionCreated, CandidateAdded, VoterRegistered, ElectionStarted, VoteCast, ElectionEnded	NoElection, PreviousElectionNotEnded, ElectionAlreadyStarted, ElectionNotStarted, ElectionAlreadyEnded, ElectionNotActive, EmptyField, NoCandidates, InvalidCandidate, InvalidAddress, AlreadyRegistered, NotRegistered, AlreadyVoted, OwnableUnauthorizedAccount

5.4 Vote Algorithm

Input: `candidateId`, sender address. **Steps:** 1) Check an election exists and has started. 2) Check it has not ended and the deadline (if any) has not passed. 3) Check the sender is registered. 4) Check the sender has not voted. 5) Check $1 \leq id \leq \text{number of candidates}$. 6) Set `hasVoted = true`. 7) Increment the candidate's `voteCount` and `totalVotes`. 8) Emit `VoteCast`. The function follows the checks-effects pattern and makes no external calls, so there is no re-entrancy surface.

```
function vote(uint256 candidateId) external onlyWhileOpen {
    uint256 id = currentElectionId;
    Voter storage v = _voters[id][msg.sender];
    if (!v.registered) revert NotRegistered();
    if (v.hasVoted) revert AlreadyVoted();
    if (candidateId == 0 || candidateId > _candidates[id].length) revert
    InvalidCandidate();

    v.hasVoted = true; // effects only, no external calls
    _candidates[id][candidateId - 1].voteCount += 1;
    totalVotes[id] += 1;
    emit VoteCast(id, msg.sender, candidateId);
}
```

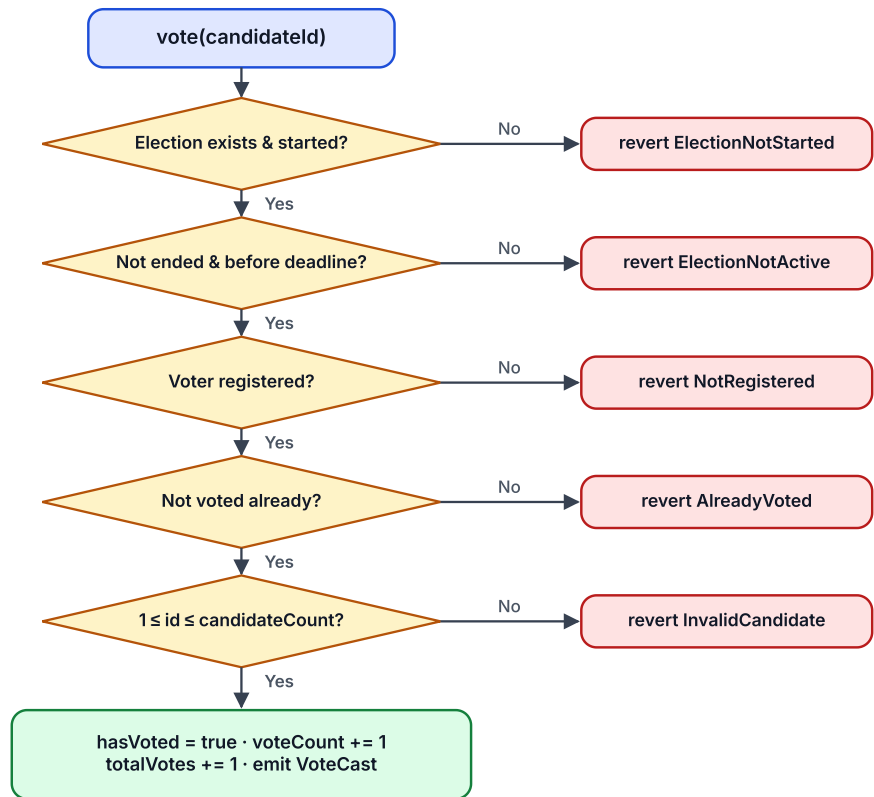


Figure 6: Vote validation flowchart inside Voting.sol

5.5 Election Life-cycle

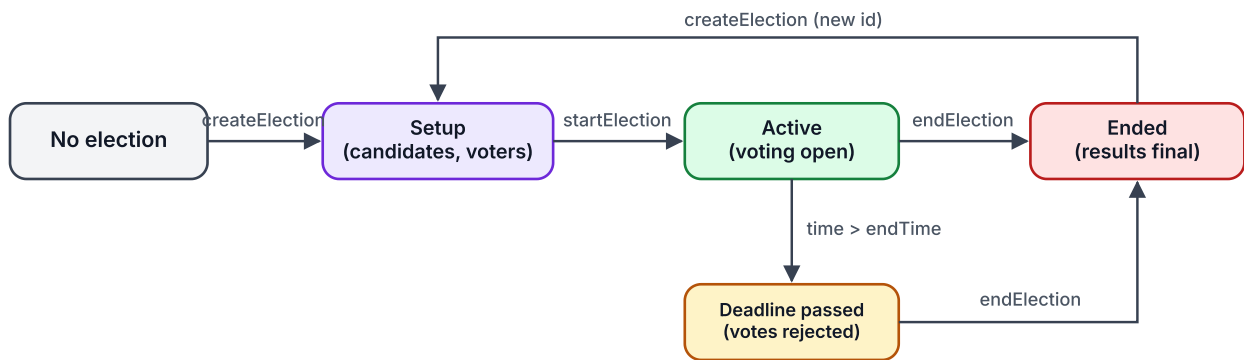


Figure 7: Election state diagram

6. Frontend Architecture

The frontend is a single-page React 18 application written in TypeScript and bundled with Vite. It is organised in layers so that only one module talks to the blockchain.

Folder / module	Responsibility
contracts/config.ts, VotingABI.json	Contract address, ABI, chain id, chain name, RPC and explorer URL from <code>.env</code> (written by the deploy script).
services/votingService.ts	All ethers.js calls: read provider, contract instances, election/candidate/voter reads, event queries, transaction details.
hooks/useWallet.tsx	MetaMask connection, auto-reconnect, accountsChanged / chainChanged, network switch / add, signer.
hooks/useElection.tsx	Election, candidates, voter status, owner, winners; polls the contract every 5 s.
hooks/useTransactions.tsx	Runs a transaction: awaiting signature → pending → confirmed/failed; toasts; refresh.
utils/errors.ts	Maps custom errors and wallet errors (rejected, insufficient gas, wrong network, unavailable contract) to friendly text.
components/	Navbar, Footer, animated network background, glass cards, candidate card, confirm modal, transaction modal, skeletons.
pages/	Home, Connect Wallet, Vote (voter dashboard), Results, How It Works, Admin, Explorer, Not Found.

Page	Main features
Home	Hero "Decentralized Voting. Transparent Results.", live election status card, how-it-works, security features, privacy note.
Connect Wallet	Address, connected vs required network, eligibility, role; handles missing MetaMask, rejection and wrong network.
Vote	Wallet, eligibility, election and voting status; candidate cards; confirmation modal; "You have already voted ✓".
Results	Registered voters, votes cast, turnout, bar chart, donut chart, ranking with progress bars, winner after the election ends.
Admin	Owner-only: create/start/end election, add candidates, batch register voters, statistics, vote counts, event history.
Explorer	Every contract event with tx hash, block, status, timestamp, from and contract address; search; transaction details.

Admin routes are hidden for non-owners in the UI, but the real protection is on-chain: even a forged request from another address is rejected with `OwnableUnauthorizedAccount`.

7. Technology Stack and System Requirements

Layer	Technology
Smart contract	Solidity 0.8.24, OpenZeppelin Contracts 5 (Ownable)
Development / test	Hardhat 2, hardhat-toolbox, Mocha + Chai, TypeScript, ethers.js v6
Network	Hardhat Local (chain id 31337) or Sepolia testnet (11155111)
Frontend	React 18, Vite 6, TypeScript 5, React Router 6
UI	Tailwind CSS 4, Lucide React icons, Recharts, react-hot-toast
Wallet / Web3	MetaMask (EIP-1193), ethers.js v6 BrowserProvider and JsonRpcProvider
Backend / database	None. All voting data lives on the blockchain.

Type	Item	Specification
Hardware	Processor / RAM	Intel Core i3 / Ryzen 3 or better; 4 GB RAM minimum, 8 GB recommended
Software	OS	Windows 10/11, Ubuntu 22.04+ or macOS
Software	Runtime	Node.js 18+ (tested on 22) and npm
Software	Browser	Chrome / Brave / Firefox with the MetaMask extension

8. Testing

8.1 Smart Contract Test Cases

The contract was tested with Hardhat using fixtures and time manipulation (`npm hardhat test`). **Result: 24 passing, 0 failing.**

ID	Feature	Expected behaviour	Result
TC01	Deployment	Deployer is owner, no election yet	Pass
TC02	Create election	ElectionCreated emitted, id = 1	Pass
TC03	Create election	Empty title / second running election rejected	Pass
TC04	Add candidate	Sequential ids, CandidateAdded emitted	Pass
TC05	Add candidate	Needs election; empty name/party rejected	Pass
TC06	Add candidate	Rejected after election started	Pass
TC07	Register voter	VoterRegistered emitted, status = registered	Pass
TC08	Register voters (batch)	5 addresses registered in one tx	Pass
TC09	Register voter	Duplicate & zero address rejected	Pass
TC10	Start election	ElectionStarted, votingOpen = true, deadline set	Pass
TC11	Start election	Without candidates / twice rejected	Pass
TC12	Successful vote	Count +1, totalVotes +1, hasVoted = true	Pass
TC13	Duplicate vote	Second vote (same or other candidate) → AlreadyVoted	Pass
TC14	Unregistered voter	NotRegistered	Pass
TC15	Vote before start	ElectionNotStarted	Pass
TC16	Vote after end	ElectionNotActive	Pass
TC17	Vote after deadline	Time travel +61 s → ElectionNotActive	Pass
TC18	Invalid candidate	id 0 and id 4 → InvalidCandidate	Pass
TC19	No election	vote() → NoElection	Pass
TC20	End election	ElectionEnded, results frozen, winner = #2	Pass
TC21	Tie	getWinners() returns [1, 3]	Pass
TC22	End election	Before start / twice rejected	Pass
TC23	New election	History of election #1 preserved	Pass
TC24	Admin authorization	All admin functions revert for non-owner	Pass

8.2 End-to-end Test and Gas Usage

An end-to-end run was performed on the local Hardhat network through the real user interface: the admin started the election, five registered voters voted (3 for Priya Patil, 1 for Aarav Sharma, 1 for Rahul Deshmukh), an unregistered wallet was blocked, the admin ended the election and the results page declared the winner. The gas used by each transaction was read from the receipts:

Transaction	Gas used
Deploy Voting.sol	1,831,518
createElection	71,620
addCandidate (first)	216,580
addCandidate	≈199,450
registerVoters (5 addresses)	176,501
startElection	76,787
vote (first vote of election)	82,623
vote (candidate already has votes)	48,423
endElection	57,210

9. Results and Output

Observed results of the local deployment:

- Network: Hardhat Local, chain id 31337. Contract address: `0x5FbDB2315678afecb367f032d93F642f64180aa3` (deployed in block #1).
- Election “Student Council Election 2026” with candidates Aarav Sharma (Student First), Priya Patil (Future Forward) and Rahul Deshmukh (Progress Alliance).
- 5 registered voters, 5 votes cast, 100% turnout. Final result: Priya Patil 3 votes (60%), Aarav Sharma 1 (20%), Rahul Deshmukh 1 (20%). Winner: Priya Patil.
- Duplicate vote, unregistered wallet and non-owner admin access were all rejected.

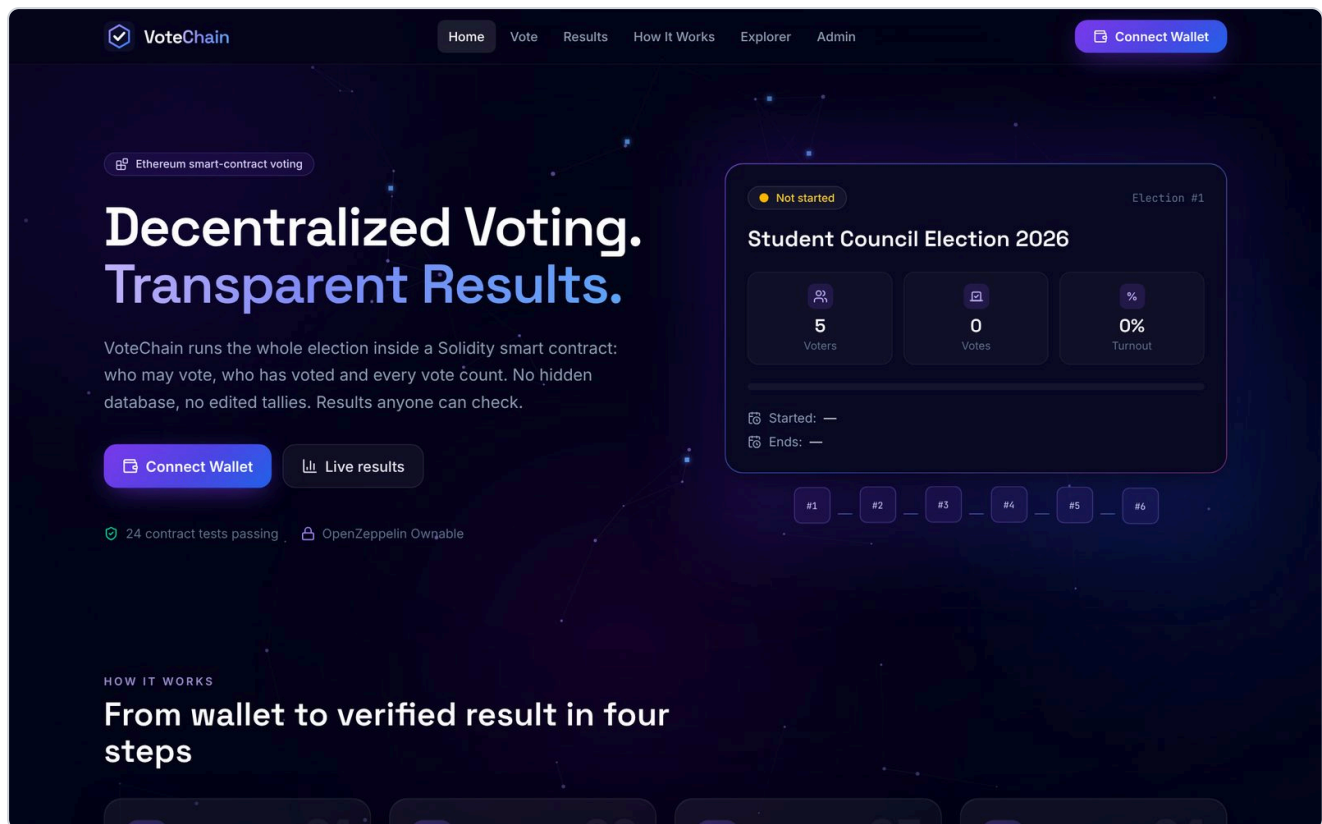


Figure 8: Landing page with live election status card

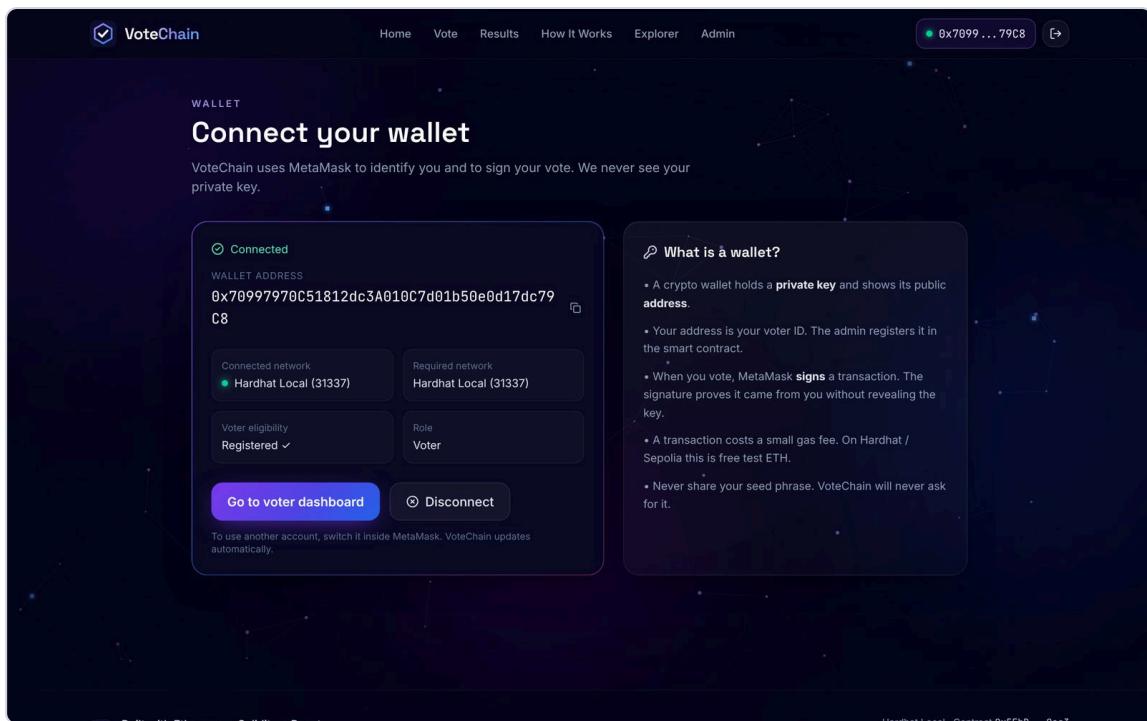


Figure 9: Connect Wallet page: address, network, eligibility and role

The screenshot displays the 'Admin Dashboard' of the VoteChain system, titled 'Election control centre'. It features a navigation bar with links to Home, Vote, Results, How It Works, Explorer, and Admin. A user address '0xf39f...2266' is shown in the top right. The dashboard includes several key sections:

- Summary Cards:**
 - Election:** #1, Student Council Election 2026
 - Registered voters:** 5
 - Votes cast:** 0, 0% turnout
 - Candidates:** 3
- Election lifecycle:**
 - 1 - Create election: Form with 'Student Council Election 2026' and a 'Create' button.
 - 2 - Start voting (optional duration in minutes, 0 = until ended manually): Form with '0' and a 'Start' button.
 - 3 - End election and freeze results: Form with an 'End election' checkbox.
- Add candidate:** Form with fields for 'Candidate name', 'Party / group', and 'Short manifesto', followed by an 'Add candidate on-chain' button.
- Register voters:** Form for pasting wallet addresses, followed by a 'Register voters' button.
- Candidate vote counts:** A table showing the following data:

ID	NAME	PARTY	VOTES	%
1	Aarav Sharma	Student First	0	0
2	Priya Patil	Future Forward	0	0
3	Rahul Deshmukh	Progress Alliance	0	0
- Transaction history / events:** A list of events including voter registrations, candidate additions, election creation, and ownership transfer, each with a timestamp and a link to the explorer.

At the bottom, it states 'Built with Ethereum + Solidity + React' and 'Blockchain-powered • Transparent • Tamper-resistant'. Footer information includes 'Hardhat Local • Contract 0x5FbD...8aa3' and 'Academic demonstration project • Yashvant Dayanand Mane (F24121004)'.

Figure 10: Admin dashboard before the election starts (candidates and voters registered on-chain)

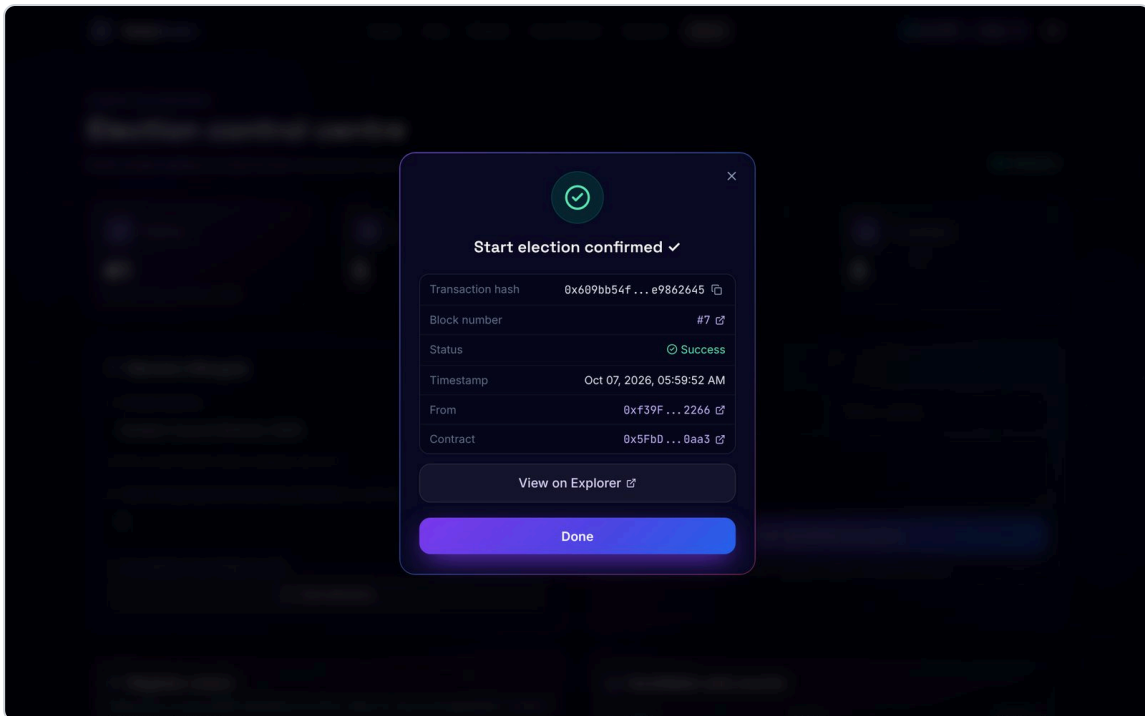


Figure 11: Admin starts the election: confirmed transaction with hash, block, status, timestamp, sender and contract

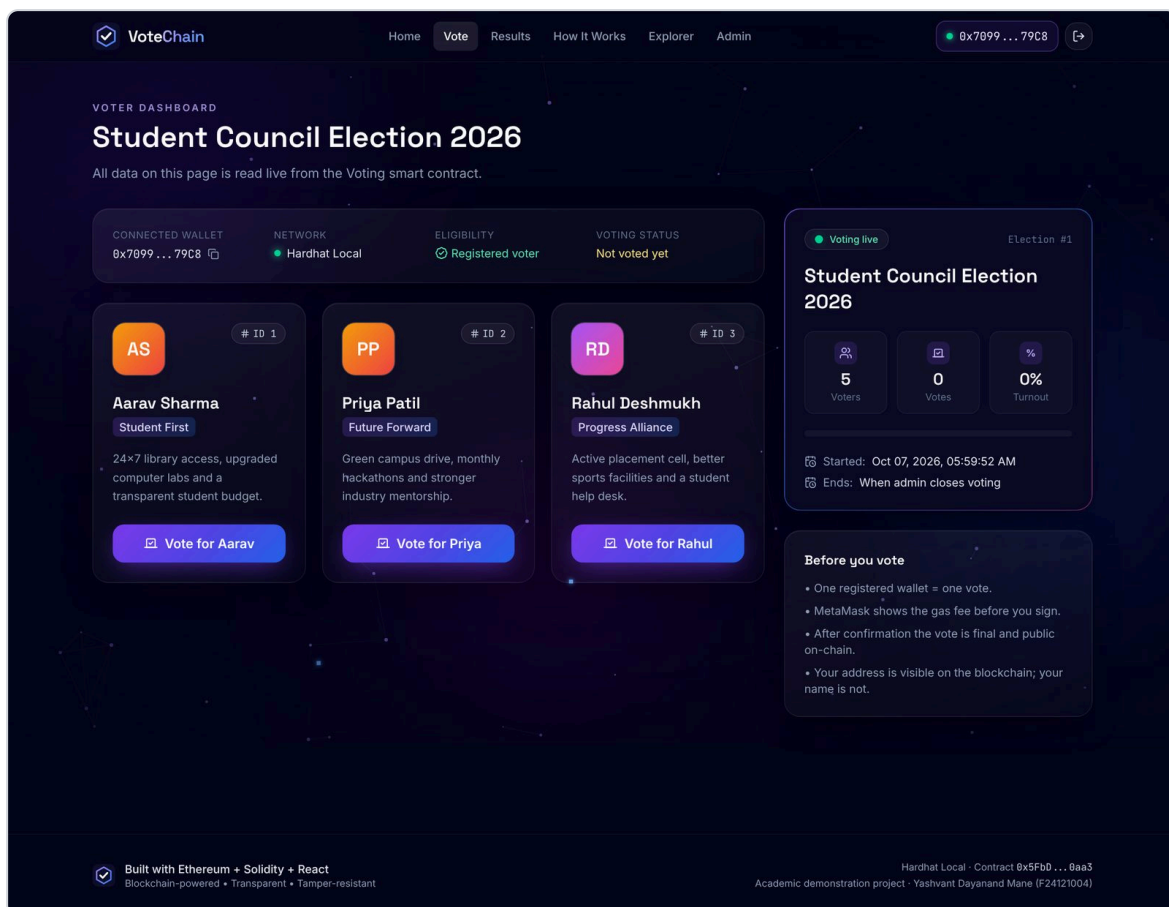


Figure 12: Voter dashboard with candidate cards (photo/avatar, name, party, manifesto, id, vote button)

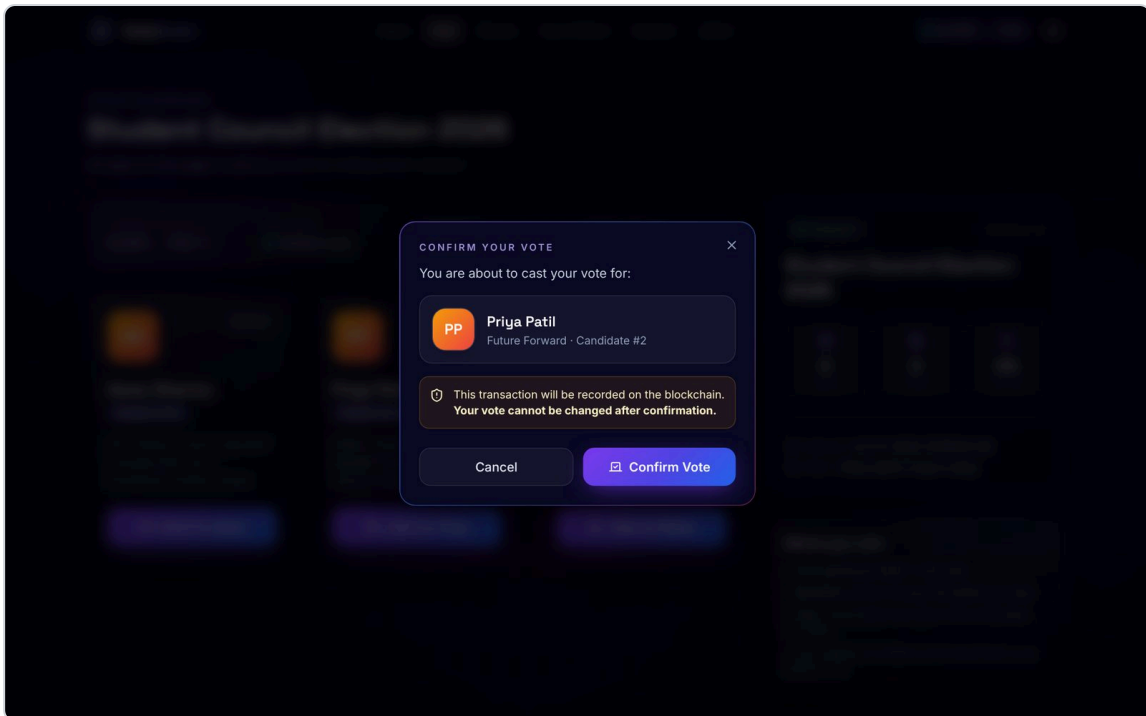


Figure 13: Vote confirmation modal

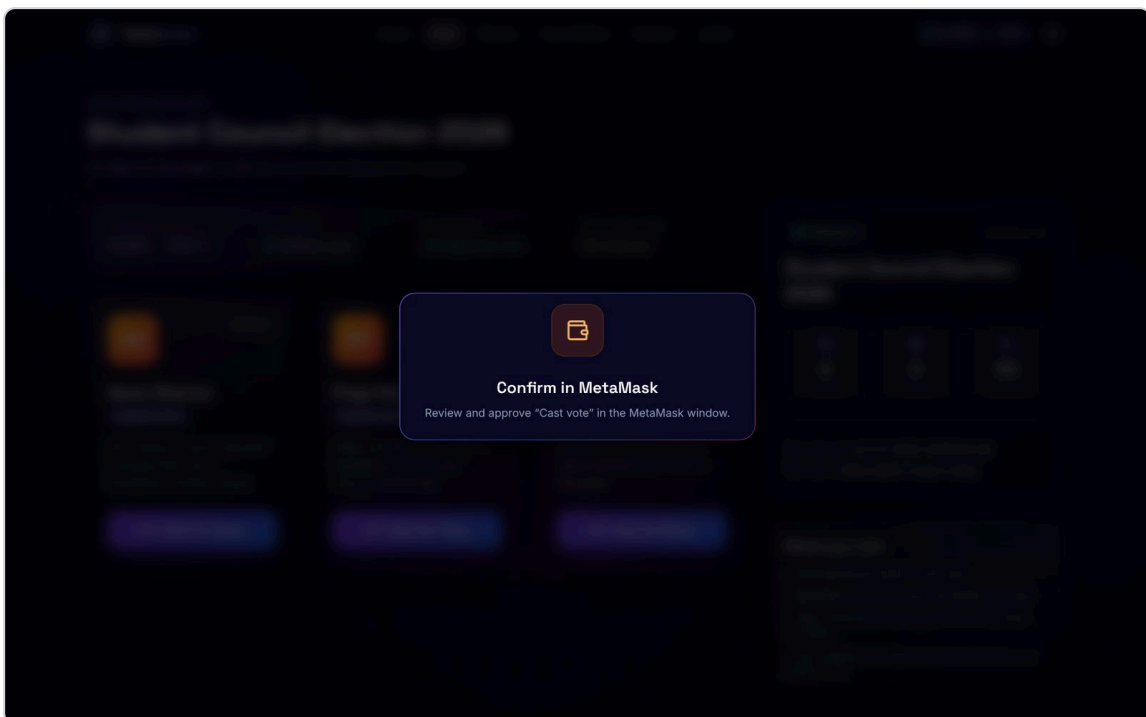


Figure 14: Waiting for confirmation in MetaMask

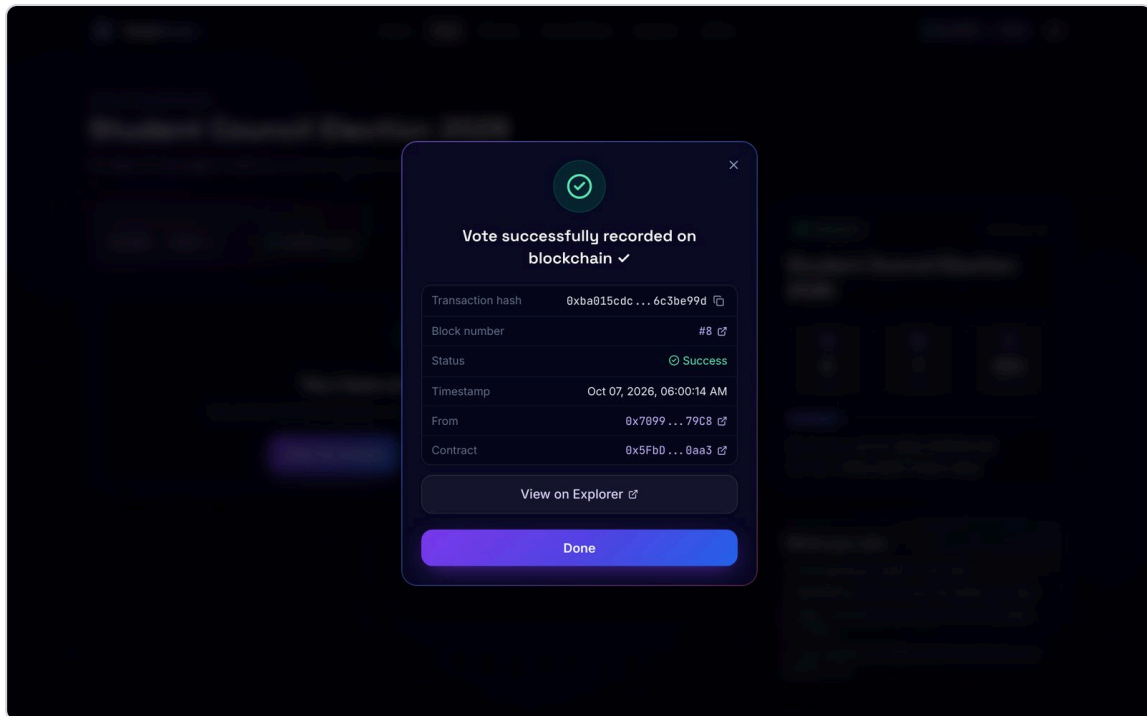


Figure 15: "Vote successfully recorded on blockchain ✓" with transaction hash and explorer link

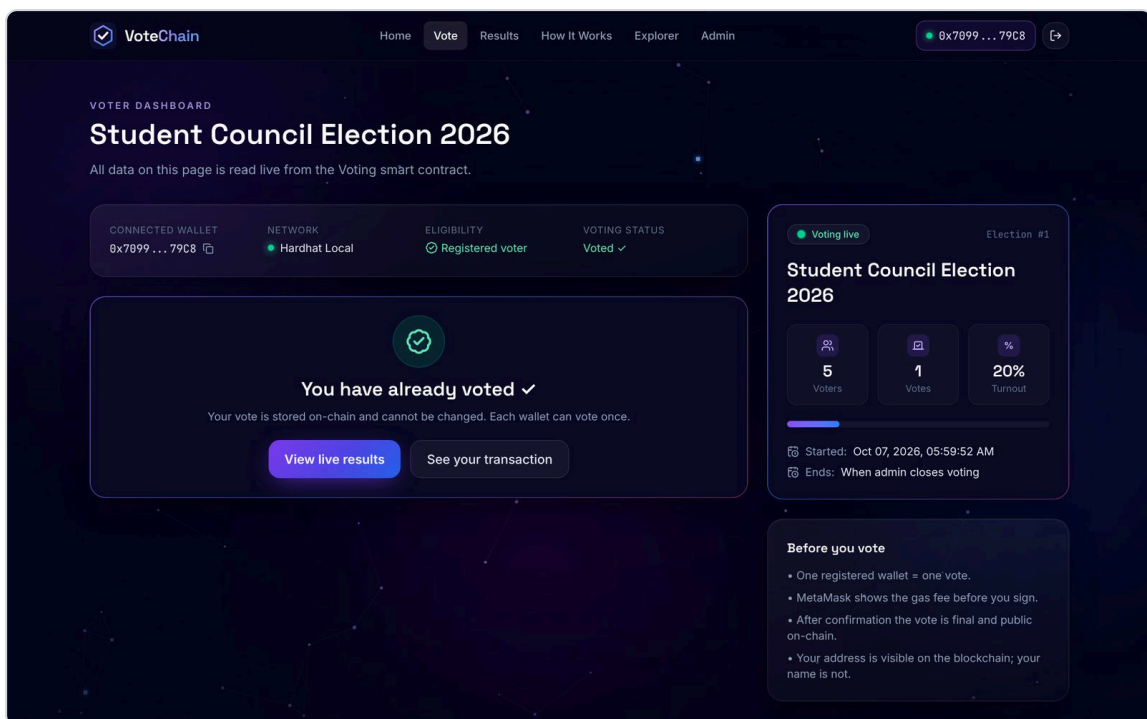


Figure 16: After voting: "You have already voted ✓"

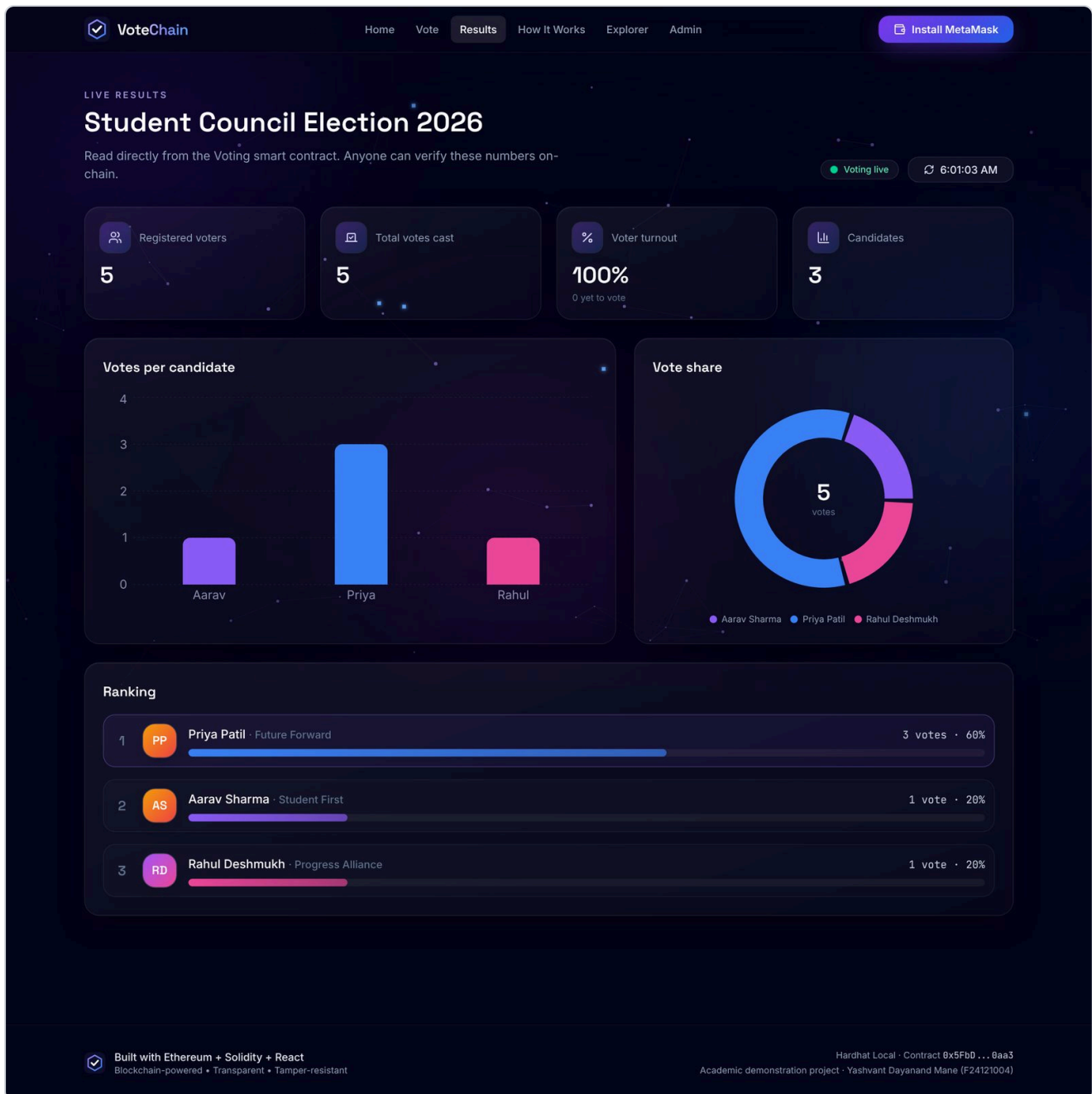


Figure 17: Live results while voting is open (bar chart, donut chart, ranking)



Figure 18: Unregistered wallet: voting disabled with "Not registered"

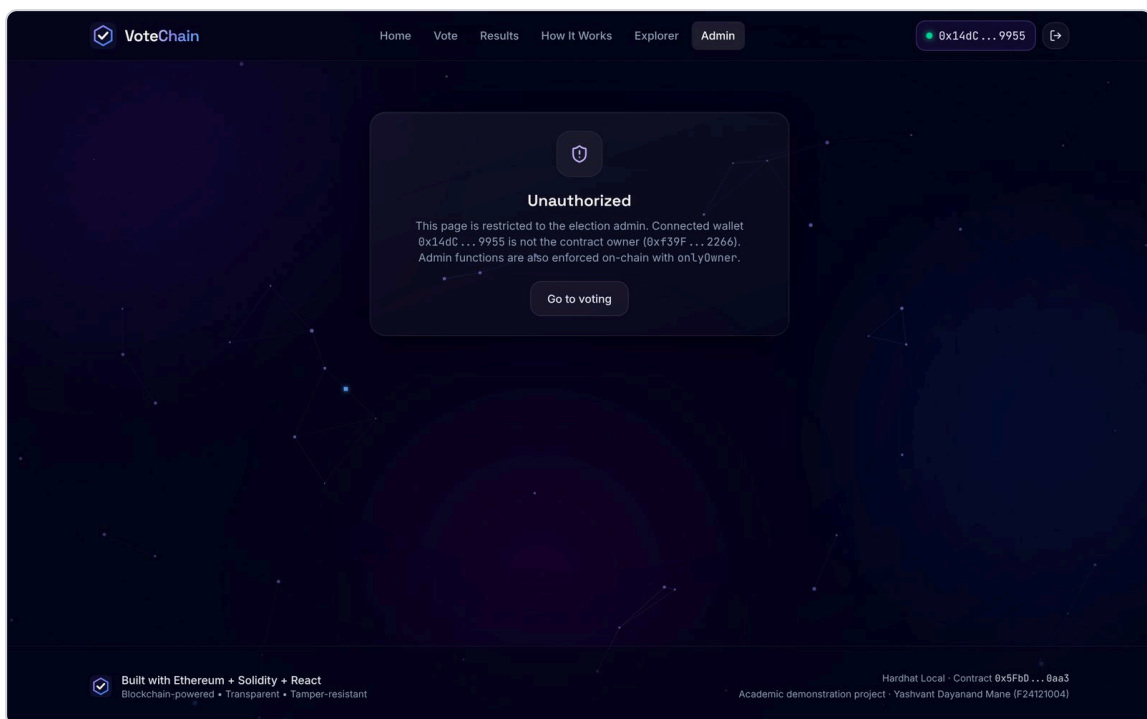


Figure 19: Non-owner wallet blocked from the admin dashboard

VoteChain

[Home](#)
[Vote](#)
[Results](#)
[How It Works](#)
[Explorer](#)
[Admin](#)

0xf39f...2266

BLOCKCHAIN EXPLORER

Every action, on the record

All events emitted by the Voting contract 0x5FbD...0aa3 on Hardhat Local. Local chain: details are fetched directly from the node.

All
VoteCast
VoterRegistered
CandidateAdded
ElectionCreated
ElectionStarted
ElectionEnded

EVENT	TX HASH	BLOCK	STATUS	TIMESTAMP	FROM	CONTRACT
VoteCast 0x9965...A4dc voted for candidate #3	0xfddb92a2...e54d648d	#12	Success	Oct 07, 2026, 06:01:02 AM	0x9965...A4dc	0x5FbD...0aa3 View on Explorer
VoteCast 0x15d3...6A65 voted for candidate #2	0x4e701e3a...2a9ac80a	#11	Success	Oct 07, 2026, 06:00:51 AM	0x15d3...6A65	0x5FbD...0aa3 View on Explorer
VoteCast 0x90F7...b906 voted for candidate #1	0x565b117f...416a4182	#10	Success	Oct 07, 2026, 06:00:41 AM	0x90F7...b906	0x5FbD...0aa3 View on Explorer
VoteCast 0x3C44...93BC voted for candidate #2	0x3fbb399a...4f2934e4	#9	Success	Oct 07, 2026, 06:00:31 AM	0x3C44...93BC	0x5FbD...0aa3 View on Explorer
VoteCast 0x7099...79C8 voted for candidate #2	0xba015cdc...6c3be99d	#8	Success	Oct 07, 2026, 06:00:14 AM	0x7099...79C8	0x5FbD...0aa3 View on Explorer
ElectionStarted Voting opened	0x609bb54f...e9862645	#7	Success	Oct 07, 2026, 05:59:52 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
VoterRegistered 0x9965...A4dc registered	0x969b1ab3...6984bfcc	#6	Success	Oct 07, 2026, 05:59:38 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
VoterRegistered 0x15d3...6A65 registered	0x969b1ab3...6984bfcc	#6	Success	Oct 07, 2026, 05:59:38 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
VoterRegistered 0x90F7...b906 registered	0x969b1ab3...6984bfcc	#6	Success	Oct 07, 2026, 05:59:38 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
VoterRegistered 0x3C44...93BC registered	0x969b1ab3...6984bfcc	#6	Success	Oct 07, 2026, 05:59:38 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
VoterRegistered 0x7099...79C8 registered	0x969b1ab3...6984bfcc	#6	Success	Oct 07, 2026, 05:59:38 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
CandidateAdded Rahul Deshmukh (Progress Alliance) added as #3	0xf2e2b707...957ff965	#5	Success	Oct 07, 2026, 05:59:37 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
CandidateAdded Priya Pali (Future Forward) added as #2	0xa42373b1...fcfaad64	#4	Success	Oct 07, 2026, 05:59:36 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
CandidateAdded Aarav Sharma (Student First) added as #1	0x5be9f44f...7ee01783	#3	Success	Oct 07, 2026, 05:59:35 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
ElectionCreated "Student Council Election 2026" created	0xc4fda247...957fbf3c	#2	Success	Oct 07, 2026, 05:59:34 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer
OwnershipTransferred Owner set to 0xf39f...2266	0xb3df2af2...c3167129	#1	Success	Oct 07, 2026, 05:59:32 AM	0xf39f...2266	0x5FbD...0aa3 View on Explorer

Built with Ethereum + Solidity + React
Blockchain-powered • Transparent • Tamper-resistant

Hardhat Local • Contract 0x5FbD...0aa3
Academic demonstration project • Yashvant Dayanand Mane (F24121004)

Figure 20: Built-in blockchain explorer listing every contract event





Figure 23: Final results with the winner declared after the election ended

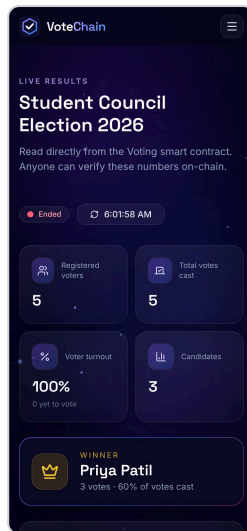


Figure 24: Responsive mobile layout

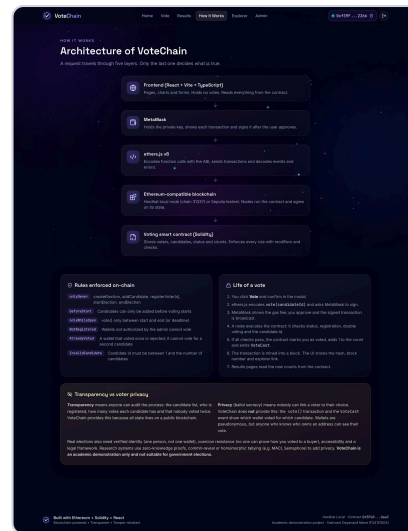


Figure 25: How It Works page: architecture and privacy explanation

10. Security Considerations, Transparency and Privacy

10.1 Security Measures

- **Access control:** OpenZeppelin `Ownable` ; every admin function is `onlyOwner` .
- **Phase control:** `beforeStart` and `onlyWhileOpen` modifiers, plus an optional time-based deadline using `block.timestamp` .
- **Input validation:** empty fields, zero address, duplicate registration and out-of-range candidate ids revert.
- **One vote per wallet:** `hasVoted` is set before the count is updated; a second call reverts with `AlreadyVoted` .
- **No ether handling and no external calls** in `vote()` , removing re-entrancy and fund-loss risks.
- **Gas efficiency:** custom errors, `calldata` parameters, batch voter registration, optimizer enabled (200 runs).
- **Frontend:** no private keys handled; configuration via environment variables; the wrong network is detected before signing.

10.2 Transparency versus Voter Privacy

Transparency means the process can be audited: anyone can see the candidates, the number of registered voters, the vote counts and that no address voted twice. VoteChain provides this.

Privacy (ballot secrecy) means nobody can link a voter to their choice. VoteChain does **not** provide this. The `vote(candidateId)` transaction and the `VoteCast` event reveal which wallet voted for which candidate. Wallets are pseudonymous, but anyone who knows who owns an address can see their vote. Blockchain voting is therefore **not** completely anonymous.

This is an academic demonstration. It is **not suitable for real government elections** without verified identity, privacy-preserving cryptography, coercion resistance, independent audits and a legal framework.

11. Limitations and Future Enhancements

11.1 Limitations

- Votes are linkable to wallet addresses (no ballot secrecy) and a voter could prove their vote to a third party (no coercion resistance).
- One wallet represents one voter; the admin must verify off-chain that each address belongs to a distinct eligible person.
- A single owner key controls the election life-cycle; losing or leaking it is a risk.
- Every vote costs gas; on a public mainnet this would be a cost barrier for voters.
- Candidate text is stored on-chain, which is expensive for long manifestos or images.
- The frontend polls the node; very large elections would need an indexer (e.g. The Graph).

11.2 Future Enhancements

- Privacy-preserving voting with zero-knowledge proofs (Semaphore, MACI) or a commit-reveal scheme.
- Identity integration (college ID / DigiLocker based verification, soulbound voter tokens).
- Multi-signature or DAO-based administration (Gnosis Safe, AccessControl roles).
- Gasless voting with meta-transactions (EIP-2771) or account abstraction (ERC-4337) so voters need no ETH.
- IPFS storage for candidate photos and manifestos; multiple concurrent elections; ranked-choice voting.
- Deployment to a Layer-2 network (Polygon, Arbitrum, Optimism) for lower fees, with contract verification on the block explorer.

12. Installation, Deployment and Demo Guide

```
# 1. Blockchain: install, test, run node, deploy and seed demo data
cd votechain/blockchain
npm install
npx hardhat test                # 24 passing
npx hardhat node                # terminal 1: local chain on http://127.0.0.1:8545
npm run deploy:local            # terminal 2: writes frontend/.env.local + ABI
npm run seed:local              # demo election, 3 candidates, 5 voters

# 2. Frontend
cd ../frontend
npm install
npm run dev                     # http://localhost:5173
```

MetaMask setup: add a network with RPC <http://127.0.0.1:8545>, chain id [31337](#), symbol ETH. Import Hardhat account #0 (the admin/owner) and accounts #1–#5 (registered voters)

using the private keys printed by `npx hardhat node`. These are public test keys and must never be used on a real network. Account #6 or later can be used to demonstrate an unregistered voter.

Demo flow: Admin opens the Admin page and clicks Start → voters #1–#5 connect and vote → Results update live → Admin ends the election → the winner is shown. For Sepolia, set `SEPOLIA_RPC_URL` and `PRIVATE_KEY` in `blockchain/.env` and run `npm run deploy:sepolia`; explorer links then open Sepolia Etherscan.

13. Conclusion

VoteChain demonstrates how a smart contract can act as a trustworthy, shared election authority. The **Voting** contract stores voters, candidates, status and vote counts on-chain, enforces one vote per registered wallet, restricts administration to the owner and publishes an event for every action. The React dApp provides a complete user experience, from MetaMask connection and eligibility checks to vote confirmation, transaction tracking, live charts and a built-in explorer, while reading every number directly from the blockchain.

The contract was verified with 24 automated test cases and an end-to-end election on a local Hardhat network. The project also makes its limits explicit: blockchain voting provides integrity and transparency but not, by itself, privacy or real-world identity. With zero-knowledge privacy, identity integration and multi-signature administration, the same architecture could serve as the basis for trustworthy institutional elections.

Project outcome: a working decentralized e-voting application with on-chain voter registration, candidate management, one-vote enforcement, publicly verifiable results and complete transaction traceability.

14. References

1. S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
2. G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper).”
3. Solidity Documentation, v0.8.24 – <https://docs.soliditylang.org>
4. OpenZeppelin Contracts 5.x Documentation – Access Control / Ownable – <https://docs.openzeppelin.com/contracts>
5. Hardhat Documentation – <https://hardhat.org/docs>
6. ethers.js v6 Documentation – <https://docs.ethers.org/v6>
7. EIP-1193: Ethereum Provider JavaScript API – <https://eips.ethereum.org/EIPS/eip-1193>
8. MetaMask Developer Documentation – <https://docs.metamask.io>
9. React, Vite, Tailwind CSS and Recharts official documentation.
10. F. Þ. Hjálmarsson et al., “Blockchain-Based E-Voting System,” IEEE CLOUD, 2018.
11. Privacy & Scaling Explorations, “MACI: Minimal Anti-Collusion Infrastructure.”

Appendix A: Mini Project Presentation Points

1. **Title:** VoteChain – Decentralized E-Voting System · Yashvant Dayanand Mane (F24121004, Roll 68) · Guide: Dr. S. R. Khonde.
2. **Problem:** centralized e-voting requires trusting whoever controls the database.

3. **Idea:** move the election rules and the ballot box into an Ethereum smart contract.
4. **Architecture:** React → MetaMask → ethers.js → Ethereum → Voting.sol (Figure 1).
5. **Contract:** Candidate / Voter / Election structs, onlyOwner, phase modifiers, 6 events, custom errors.
6. **Rules enforced on-chain:** registered only, one vote, valid candidate, only while open.
7. **Live demo:** admin starts election → voter connects, confirms vote in MetaMask → tx hash and block → results update → duplicate vote rejected → admin ends → winner.
8. **Explorer:** every action has a transaction hash, block, timestamp and sender.
9. **Testing:** 24 Hardhat tests passing; gas per vote about 48k–83k.
10. **Honest limits:** transparency is not privacy; academic system, not for government elections.
11. **Future:** ZK privacy, identity, multisig admin, gasless voting, Layer-2.